What is SDLC?

The process of developing and delivering software projects is called SDLC

Phases:
1) Requirement Gathering
2) Requirement Analysis
3) Design / Planning
4) Development
5)Testing
6) Deployment
7) Delivery
8) Maintenance

Waterfall Methodology

-> Earlier people used to follow Waterfall Methodology to develop projects

-> Waterfall is a linear methodology to develop and deliver projects

-> Everything will happen step by step

-> If one step completed then only we will go to next step

-> We will move only in forward direction (No backward direction)

-> Requirements are fixed

-> Budget is fixed

> Client involvement is very less

-> Client will see the project at the end

Note: Waterfall Methodology is not suitable for big projects

Agile Methodology

-> Agile is an iterative approach to develop and deliver the projects

-> Development and testing will happen parallelly

-> Client involvement will be very high

-> We will deliver project in multiple releases (Sprints)

-> For every release we will take client feedback.

-> Requirements are not fixed

-> Budget is not fixed

-> Project Development, Testing & Delivery is very frequent is Agile

-> Using DevOps culture we can adopt agile methodology very easily

DevOps is promoting Agile methodology

>Using DevOps we can achieve Continuous Integration [CI] & Continuous Deployment/Delivery (CD)

What is DevOps?

-> DevOps is a culture and process which is used to deliver the projects to the client quickly

-> DevOps means set of practises

-> Using DevOps we can reduce the time of Software Development Life Cycle

-> DevOps means the collaboration between Development and Operations

DevOps Development + Operations

-> Development team is responsible to write the code as per client requirements

-> Once development is completed then Operations team is responsible to deploy and deliver that project to the client

> By Using DevOps process Development team & Operations team will work together to make sure project is getting delivered client with in given estimated time.

The Logo of DevOps represents infinate because it is a never ending continous process.

DevOps Advantages

1) Speed

2) Rapid Development

3) Quick Releases

4) Reliability

5) Security

6) Client Satisfaction

7) Teams Collaboration

Note: DevOps is not one person job, it is everyone's job in the project

DevOps tools overview

Build Tools (Ant/Maven/Gradle)

Repository Tools (SVN/GitHub/BitBucket)

Code Review Tools (PMD/Sonar Qube/Sonar lint)

Code Deployment Tools (Jenkins/UDeploy)

Containerization Tools (Docker)

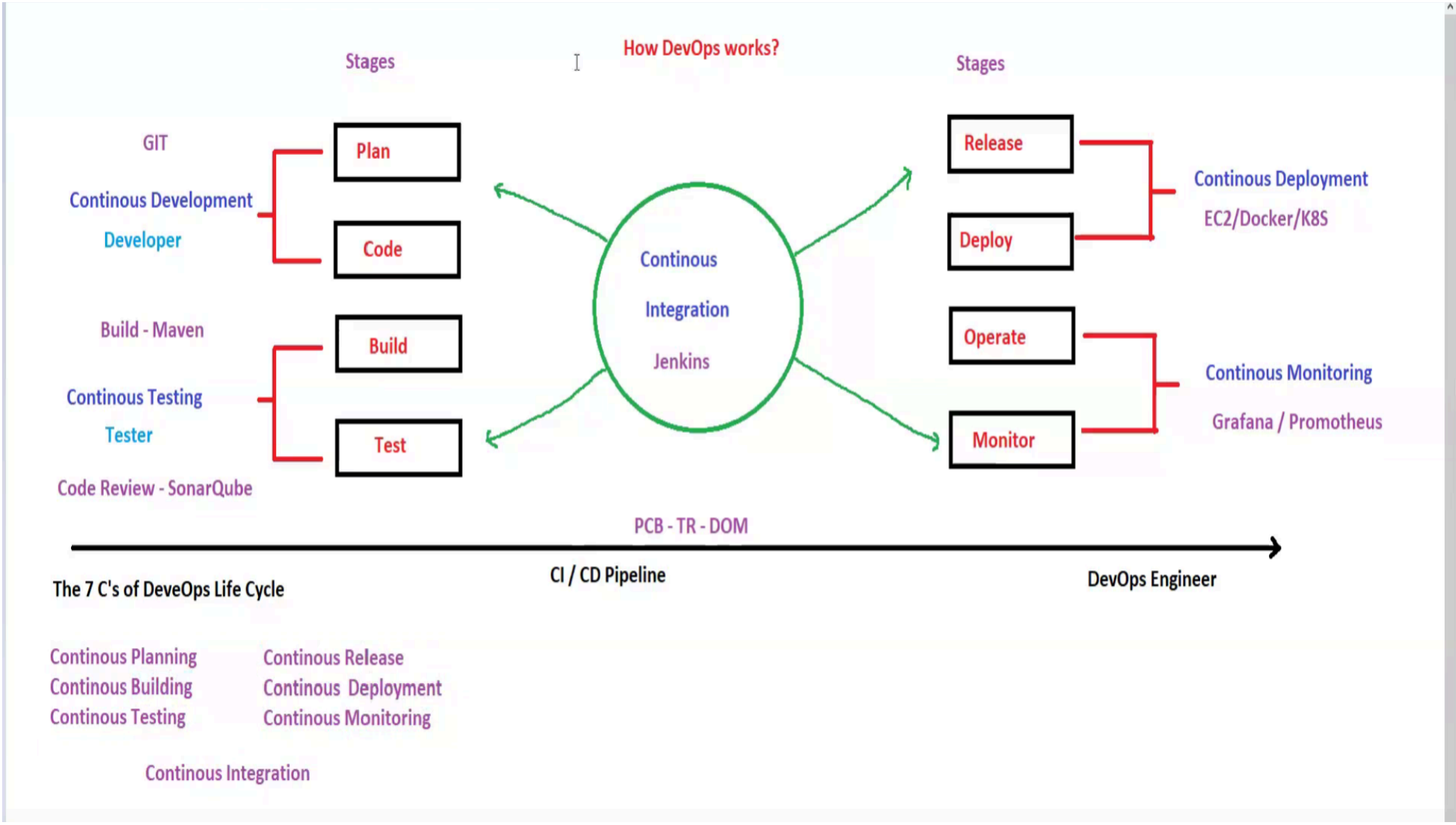
Orchestration Tools (Kubernetes)

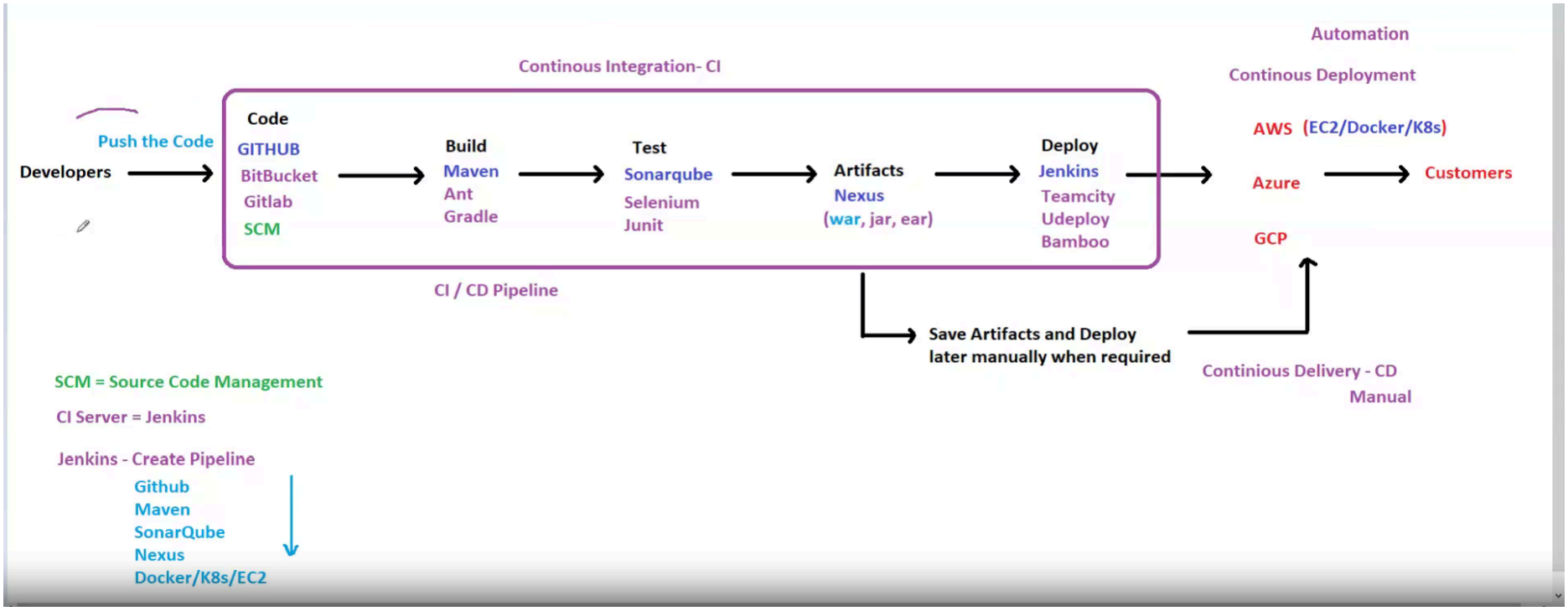
Configuration Tools (Chef/Ansible)

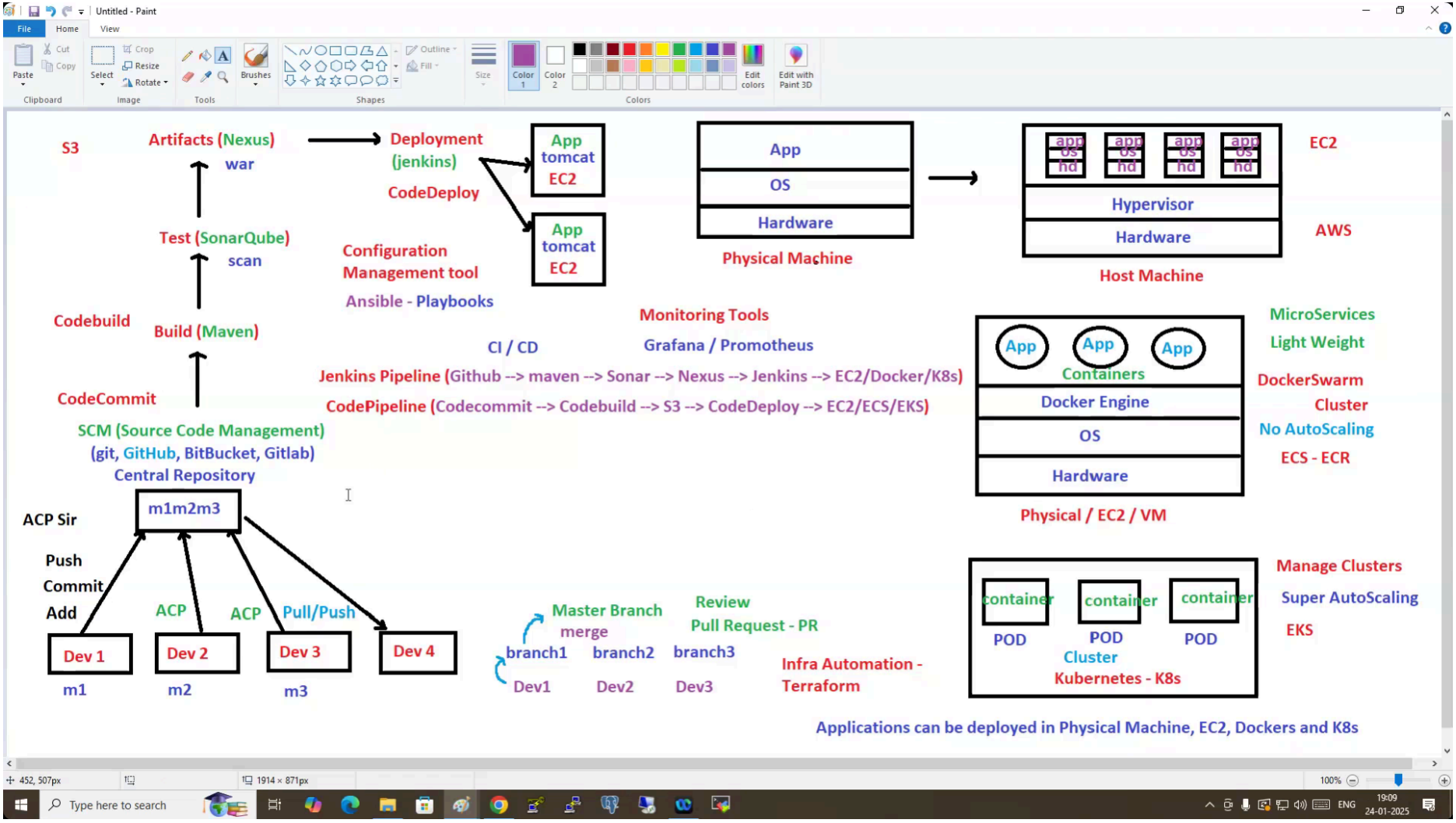
Infrastructure as a Code (laaC) (Terraform)

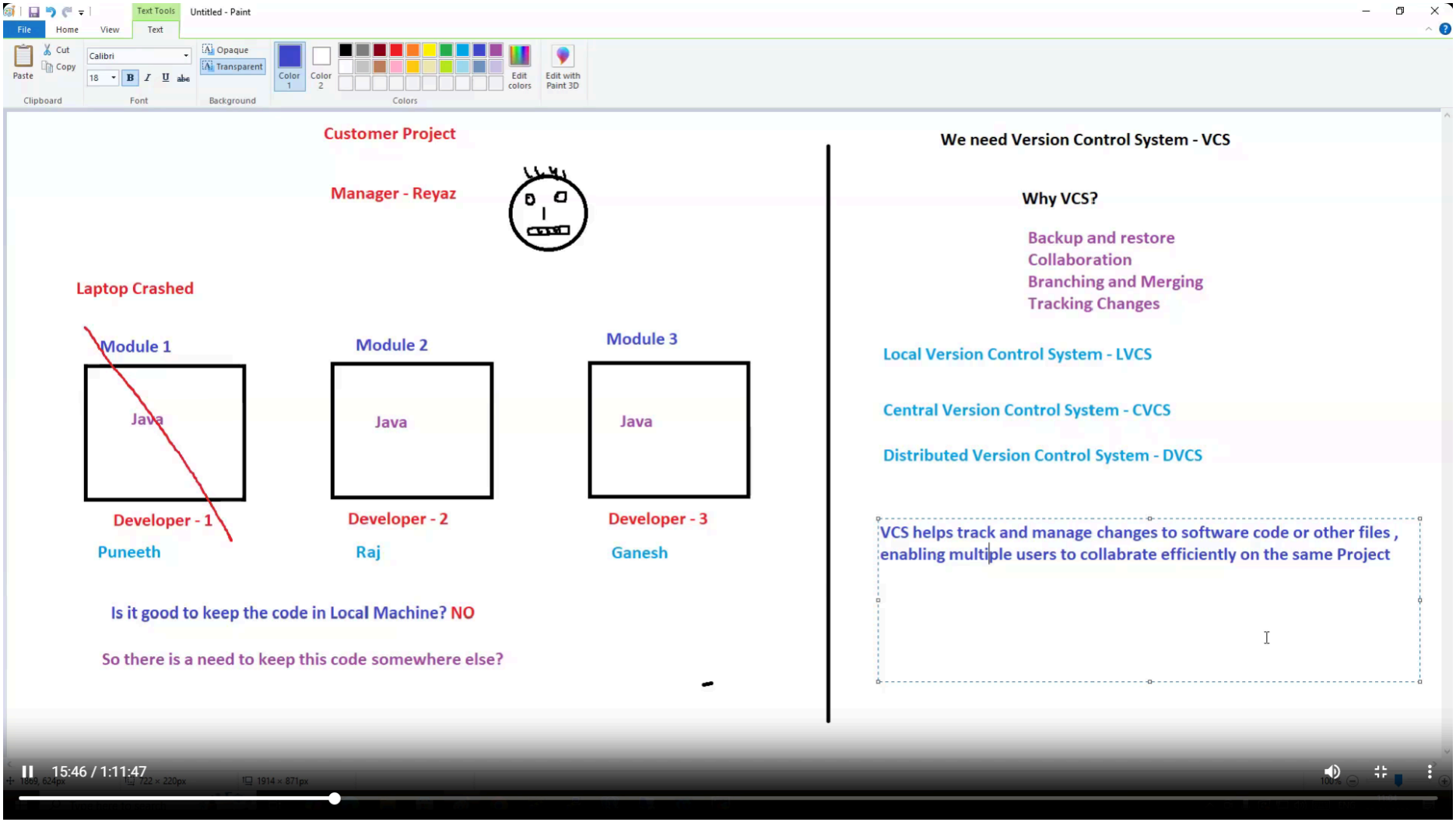
Monitoring Tools (Nagios/Promotheus Graphana)

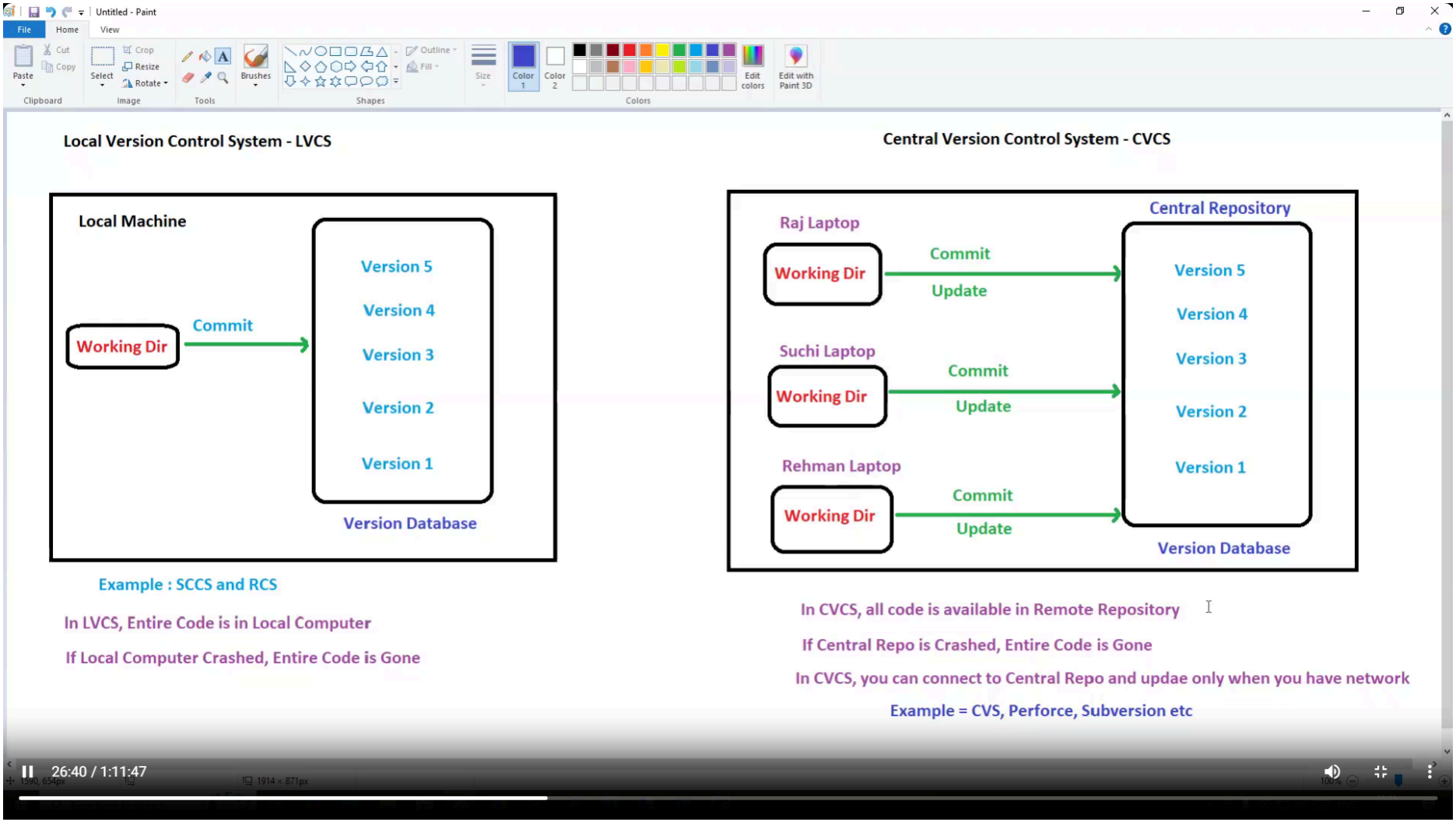
Project Management Tools (Jira/Trello)

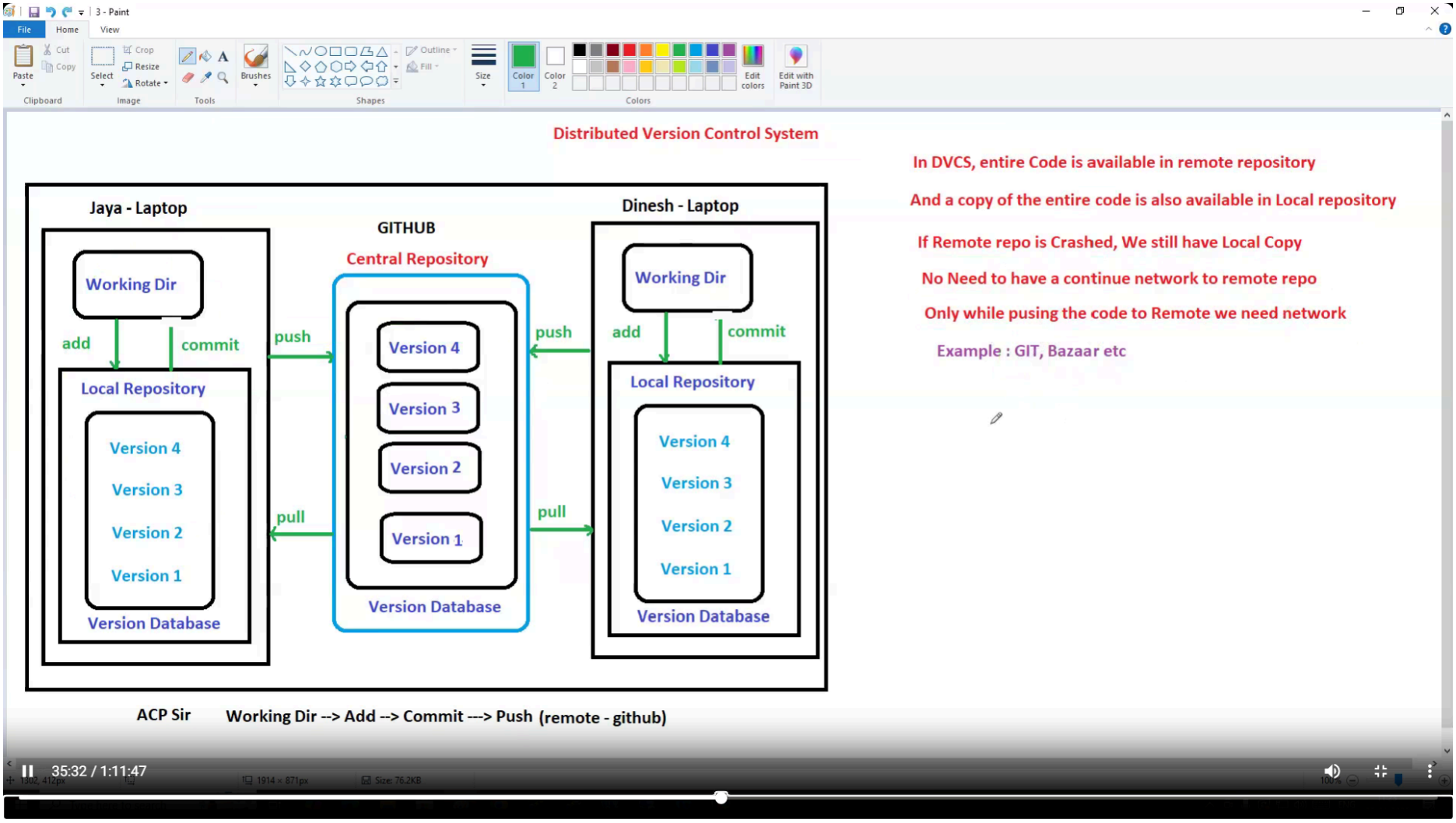

How DevOps works?
Stages
Stages
GIT
Continous Development
Developer
Plan
Code
Build - Maven
Continous Testing
Tester
Code Review - SonarQube
Build
Test
Continous
Integration
Jenkins
Release
Deploy
Operate
Monitor
Continous Deployment
EC2/Docker/K8S
Continous Monitoring
Grafana / Promotheus
PCB - TR - DOM
CI / CD Pipeline
DevOps Engineer
The 7 C's of DeveOps Life Cycle
Continous Planning
Continous Building
Continous Testing
Continous Release
Continous Deployment
Continous Monitoring
Continous Integration

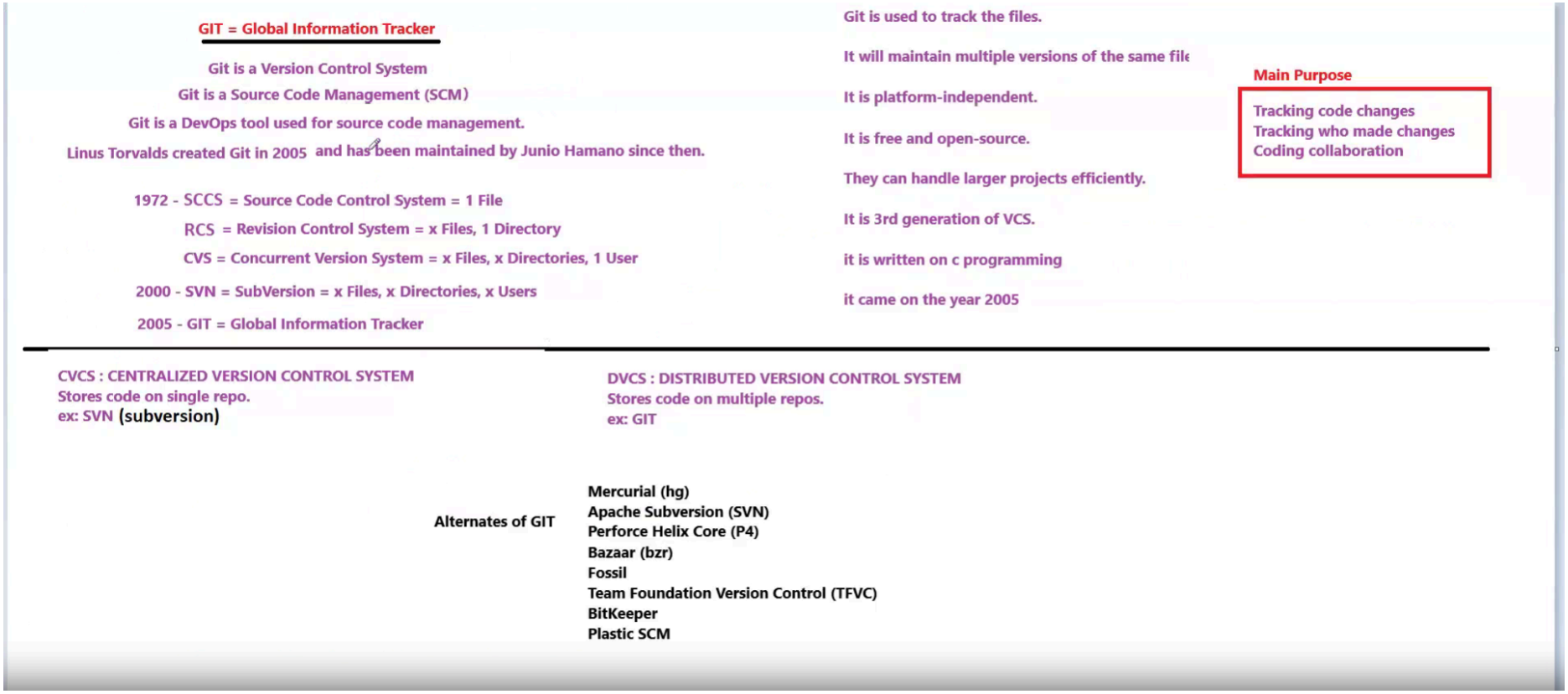

Automation
Continous Integration- CI
Continous Deployment
Push the Code
Developers
Code
GITHUB
BitBucket
Gitlab
SCM
Build
Maven
Ant
Gradle
Test
Sonarqube
Selenium
Junit
Artifacts
Nexus
(war, jar, ear)
Deploy
Jenkins
Teamcity
Udeploy
Bamboo
AWS (EC2/Docker/K8s)
Azure
GCP
Customers
CI / CD Pipeline
Save Artifacts and Deploy
later manually when required
Continious Delivery - CD
Manual
SCM = Source Code Management
CI Server = Jenkins
Jenkins - Create Pipeline
Github
Maven
SonarQube
Nexus
Docker/K8s/EC2


Untitled - Paint
S3
Artifacts (Nexus)
war
Deployment
(jenkins)
CodeDeploy
App
tomcat
EC2
App
tomcat
EC2
Configuration
Management tool
Ansible - Playbooks
Test (SonarQube)
scan
Codebuild
Build (Maven)
CodeCommit
SCM (Source Code Management)
(git, GitHub, BitBucket, Gitlab)
Central Repository
CI / CD
Monitoring Tools
Grafana / Promotheus
Jenkins Pipeline (Github --> maven --> Sonar --> Nexus --> Jenkins --> EC2/Docker/K8s)
CodePipeline (Codecommit --> Codebuild --> S3 --> CodeDeploy --> EC2/ECS/EKS)
App
OS
Hardware
Physical Machine
app
os
hd
Hypervisor
Hardware
Host Machine
EC2
AWS
App
Containers
Docker Engine
OS
Hardware
Physical / EC2 / VM
MicroServices
Light Weight
DockerSwarm
Cluster
No AutoScaling
ECS - ECR
ACP Sir
m1m2m3
Push
Commit
Add
ACP
ACP
Pull/Push
Dev 1
Dev 2
Dev 3
Dev 4
m1
m2
m3
Master Branch
merge
branch1
branch2
branch3
Dev1
Dev2
Dev3
Review
Pull Request - PR
Infra Automation -
Terraform
container
container
container
POD
POD
POD
Cluster
Kubernetes - K8s
Manage Clusters
Super AutoScaling
EKS
Applications can be deployed in Physical Machine, EC2, Dockers and K8s

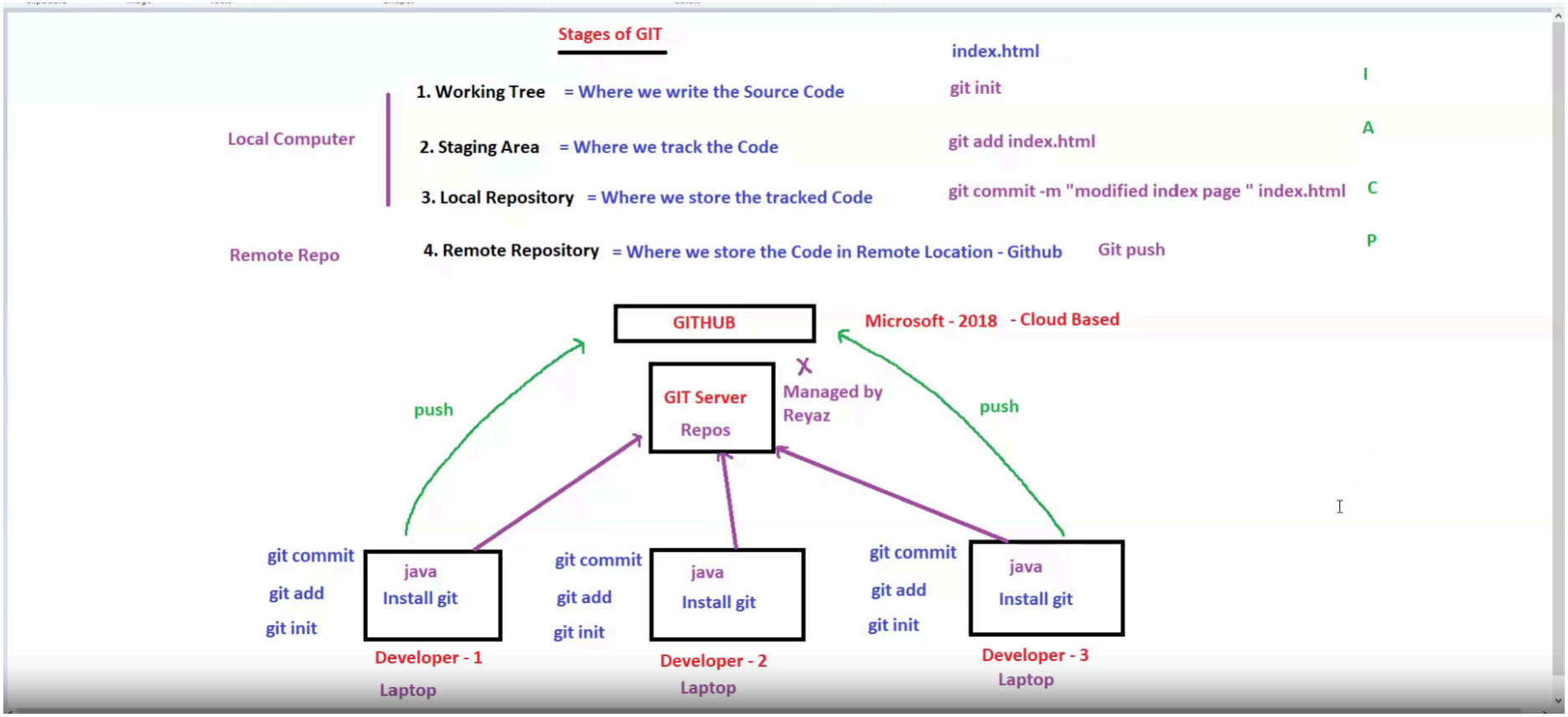

Text Tools
Untitled - Paint
File
Home
View
Text
Paste
Cut
Copy
Calibri
18
Opaque
Transparent
Color 1
Color 2
Edit colors
Edit with Paint 3D
Clipboard
Font
Background
Colors
Customer Project
Manager - Reyaz
Laptop Crashed
Module 1
Java
Developer - 1
Puneeth
Module 2
Java
Developer - 2
Raj
Module 3
Java
Developer - 3
Ganesh
Is it good to keep the code in Local Machine? NO
So there is a need to keep this code somewhere else?
We need Version Control System - VCS
Why VCS?
Backup and restore
Collaboration
Branching and Merging
Tracking Changes
Local Version Control System - LVCS
Central Version Control System - CVCS
Distributed Version Control System - DVCS
VCS helps track and manage changes to software code or other files , enabling multiple users to collabrate efficiently on the same Project
15:46 / 1:11:47


Untitled - Paint
File
Home
View
Local Version Control System - LVCS
Local Machine
Working Dir
Commit
Version 5
Version 4
Version 3
Version 2
Version 1
Version Database
Example : SCCS and RCS
In LVCS, Entire Code is in Local Computer
If Local Computer Crashed, Entire Code is Gone
Central Version Control System - CVCS
Central Repository
Raj Laptop
Working Dir
Commit
Update
Suchi Laptop
Working Dir
Commit
Update
Rehman Laptop
Working Dir
Commit
Update
Version 5
Version 4
Version 3
Version 2
Version 1
Version Database
In CVCS, all code is available in Remote Repository
If Central Repo is Crashed, Entire Code is Gone
In CVCS, you can connect to Central Repo and updae only when you have network
Example = CVS, Perforce, Subversion etc
26:40 / 1:11:47


3 - Paint
File
Home
View
Distributed Version Control System
Jaya - Laptop
GITHUB
Central Repository
Dinesh - Laptop
Working Dir
add
commit
Local Repository
Version 4
Version 3
Version 2
Version 1
Version Database
push
pull
In DVCS, entire Code is available in remote repository
And a copy of the entire code is also available in Local repository
If Remote repo is Crashed, We still have Local Copy
No Need to have a continue network to remote repo
Only while pusing the code to Remote we need network
Example : GIT, Bazaar etc
ACP Sir
Working Dir --> Add --> Commit ---> Push (remote - github)
35:32 / 1:11:47



$ mv -- *.html sanket/
Moving all html files from current directory to sub directory


Stages of GIT
index.html
1. Working Tree = Where we write the Source Code
git init
I
Local Computer
2. Staging Area = Where we track the Code
git add index.html
A
3. Local Repository = Where we store the tracked Code
git commit -m "modified index page " index.html
C
Remote Repo
4. Remote Repository = Where we store the Code in Remote Location - Github
Git push
P
GITHUB
Microsoft - 2018 - Cloud Based
X
GIT Server
Repos
Managed by
Reyaz
push
push
git commit
git add
git init
java
Install git
Developer - 1
Laptop
git commit
git add
git init
java
Install git
Developer - 2
Laptop
git commit
git add
git init
java
Install git
Developer - 3
Laptop

touch python (1..10)

touch hello.txt

git add .
git commit m "python files".
git remote add origin https://github.com/ReyazShaik/testrepo.git > copy this command from GitHub --> this will be done only first time
once above command executed, in .git folder, cat config file, remote origin got added --> show this file
git push -u origin master -> this will be used to commit the changes from local to central repo
git username:
git password or personal token [generate the token from github] ghp_Dkokvo1xBAKIGCm71LP90RnDd10Wiq118reu
git config global credential.helper cache >not to ask for the password every time
git credential-cache exit --> TO remove credentials
git remote -v
Example
Now refresh the GitHub page and you can see the files
-->open hello. Ixt and add some content here
-> git status -> now you can see the file in red color which is modified, now you need to stage this file
--> git add hello.txt -> now you have stagged this file, if you want all the files to stage use
git add-

To generate SSH Key

ssh-keygen (give this command into your terminal)
copy the publickey id_rsa.pub
paste into github ssh keys
path settings-->ssh and GPG keys--> Add new ssh key and give any name and paste it your publickey over there
cd /home/ec2-user/
git remote add origin git@github.com: ReyazShaik/test1.git
git remote -v -> to check the remote repo details

git push u origin master

Git commands learned
==========
init
add
commit
log
show
Restore

yum install -y git

git -v

.git - hidden dir

git init

touch index.html

vi index.html

git add filename

git commit -m "msg" filename

git status

git log

git log --oneline

git log --oneline -2

git show commitid

git log -p -2

git log --stat

git shortlog

git blame filename

git diff commitid..commitid

git diff

git diff --staged

git diff HEAD

git config user.name "name"

git config user.email "emailid"

git log --oneline --author='name'

git commit --amend -m "msg" = change the latest commit msg

git rebase -i HEAD~3 = change/modify multiple commit msgs

rebase - reword / squash

reword = change the commit msg

squash = combine the commit msgs

git add . / git add --a / git add *

git commit -m "msg" . or *

rm -rf *
git add .
git commit -m "clean" . } clean

git checkout commitid -- filename = go to previous version file

git checkout master -- * = again go to latest version

git rm filename = delete the file

git checkout HEAD -- filename = restore the deleted file

git restore filename = restore the file before commit

git restore --staged filename = restore from stage are to wrk dir / to untrack

git restore --staged .

git rm --cached filename

rm -rf filename

git restore filename

git reset --soft HEAD^ = This will bring from Stage3 to Stage2

git restore --staged filename

git reset --hard HEAD^ = This will bring from Stage3 to Stage1- change in commit history

git stash = hold the working task

git stash apply = restore the working task

git branch

git remote add origin urllink

git push -u origin master / git push origin master / git push

git config --global credential.helper cache

git fetch

git pull

git clone url

Token and SSH KEYS to authenticate

```
git branch
git branch branchname
git checkout branchname
git merge branchname
git revert branchname
git rebase branchname
git checkout -b branchname
git commit -am "msg"
git branch -m currentbranch newbranch
git clean -f filename
git cherry-pick commitid
git tag tagname commitid
vi .gitignore
git branch -D branchname
git reflog
git checkout -b branchname commitid
```

Maven - is build Automation tool

- Maven is project Management tool
- Maven is a Build automation tool
- Maven is for Build Purpose
- Maven will download and install necessary dependencies
- Maven has POM.xml file
- Project Object Model

POM file contains:

- Dependencies
- Plugins
- Configuration
- Project details

Build Lifecycles
Build Phases
    Validate
    Compile
    Test
    Package
    Install
    Deploy

java - Basic Raw
.class-Executable file
Jar/war-Artifacts
jar = group of class files (works for backend)
war-It contains FE and BE
FE-html, css, js, BE-java

MAVEN

Maven is a build automation and project management tool primarily used in Java-based projects

It helps manage a project's build process, dependencies, documentation, and reporting.

Maven uses a Project Object Model (POM) to define a project's structure and configuration, making it easier to handle large and complex projects.

The pom.xml file is the core of a Maven project. It defines project details, dependencies, plugins, and configurations.

Group ID: Identifies the project's group (e.g., com.example).
Artifact ID: The name of the project or module.
Version: The project's version number.
Dependencies: Maven simplifies dependency management by automatically downloading and managing project libraries from a central repository

Build Lifecycle:
Maven defines a series of build phases that must be executed in order to build, test, and deploy a project. Key phases include:

validate: Check if the project is correct and all necessary information is available.
compile: Compile the source code.
test: Run unit tests.
package: Package the compiled code into JAR or WAR format.
install: Install the package into the local repository.
deploy: Deploy the package to a remote repository for sharing.

Plugins:
Plugins extend Maven's functionality, performing tasks such as compilation, testing, packaging, deployment, and documentation generation. Popular Maven plugins include:

maven-compiler-plugin: Compiles the project's source code.
maven-surefire-plugin: Runs tests.
maven-jar-plugin: Packages the project into a JAR file.

Common Maven Commands:

mvn compile: Compiles the source code of the project.
mvn test: Runs the tests using a testing framework like JUnit.
mvn package: Packages the compiled code into a JAR/WAR file.
mvn install: Installs the project's JAR/WAR file into the local repository.
mvn deploy: Deploys the built project to a remote repository for sharing with other developers.
mvn clean: Deletes the target directory (where compiled code and other build artifacts are stored).

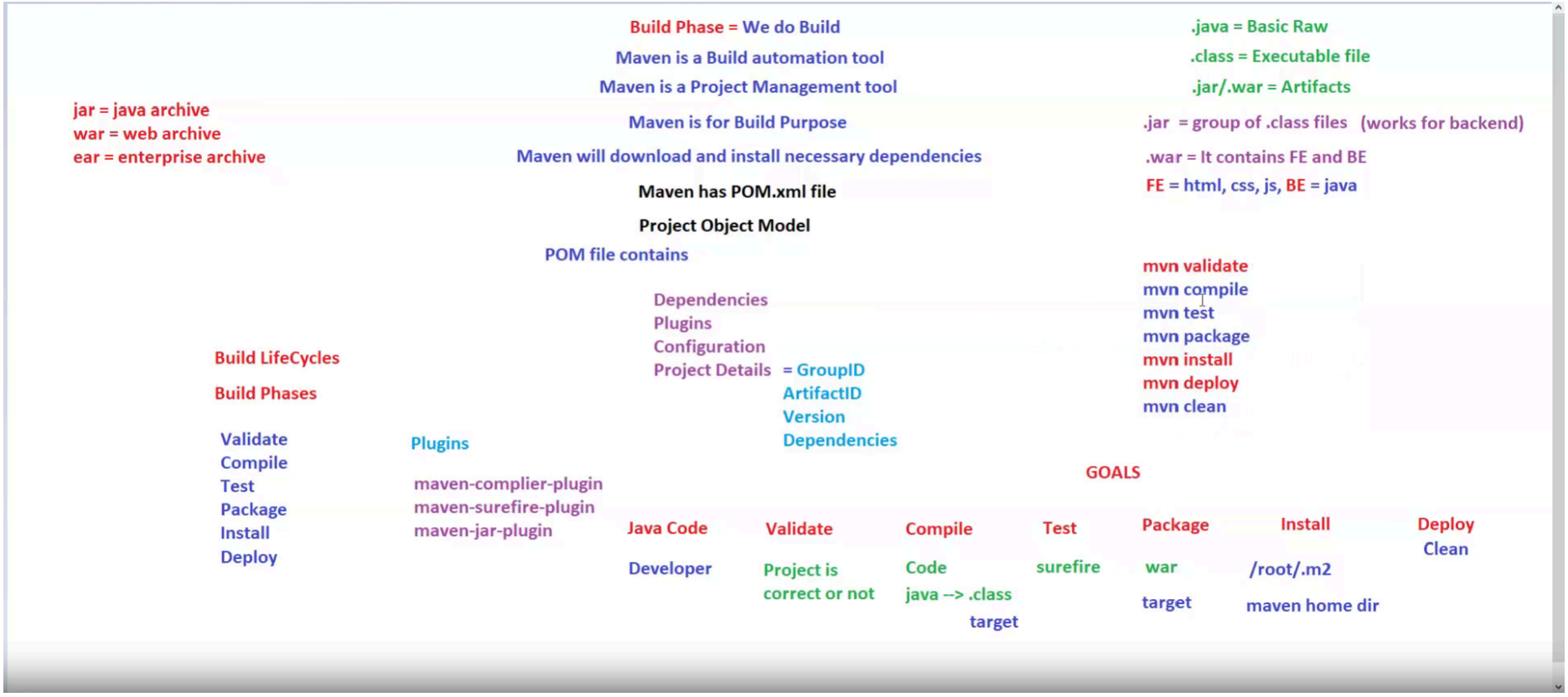
Java-Based Build Tools
Apache Maven, Gradle, Ant

Node.js Build Tools
npm, Webpack, Parcel, Grunt, Gulp, Rollup

C/C++ Build Tools
Make , CMake, Meson, Ninja,

Python Build Tools
Setuptools, Poetry, Build

NET Build Tools
MSBuild, Cake, Nuke


Build Phase = We do Build
Maven is a Build automation tool
Maven is a Project Management tool
Maven is for Build Purpose
Maven will download and install necessary dependencies
Maven has POM.xml file
Project Object Model
POM file contains
Dependencies
Plugins
Configuration
Project Details
= GroupID
ArtifactID
Version
Dependencies
.java = Basic Raw
.class = Executable file
.jar/.war = Artifacts
.jar = group of .class files (works for backend)
.war = It contains FE and BE
FE = html, css, js, BE = java
jar = java archive
war = web archive
ear = enterprise archive
mvn validate
mvn compile
mvn test
mvn package
mvn install
mvn deploy
mvn clean
Build LifeCycles
Build Phases
Validate
Compile
Test
Package
Install
Deploy
Plugins
maven-complier-plugin
maven-surefire-plugin
maven-jar-plugin
GOALS
Java Code
Developer
Validate
Project is correct or not
Compile
Code
java --> .class
target
Test
surefire
Package
war
target
Install
/root/.m2
maven home dir
Deploy
Clean

Building an Pipeline

Example Pipeline:

```
pipeline {
    agent any {
        stages{
            stage('CLONE REPO GIT')
            {
                step
                {
                    echo 'Cloning the git repository'

                }
            }
            stage('Build'){
                step{
                    echo 'Building the application from code SCM'
                }
            }
            stage('Test')
            {
                step{
                    echo 'Testing the applicatioin - SonarQube'
                }
            }
            stage('Code Review')
            {
                step{
                    echo 'Code Review Quality check - Code Review'
                }

            }
            stage('Artifacts')
            {
                step{
                    echo 'Pushing to Nexus / S3'
                }
            }
            stage('Deploy')
            {
                step{
                    echo 'Deploying the application to Tomcat'
                }
            }
        }
    }
```

What is SonarQube?

An open-source platform for continuous code quality and security.

Helps developers and teams identify bugs, vulnerabilities, code smells, and duplications.

Supports 20+ programming languages (e.g., Java, JavaScript, Python, C#)

Why Use SonarQube?

Improves code quality and readability.

Identifies potential vulnerabilities early in the development cycle.

Promotes best coding practices.

Enhances collaboration between development and security teams.

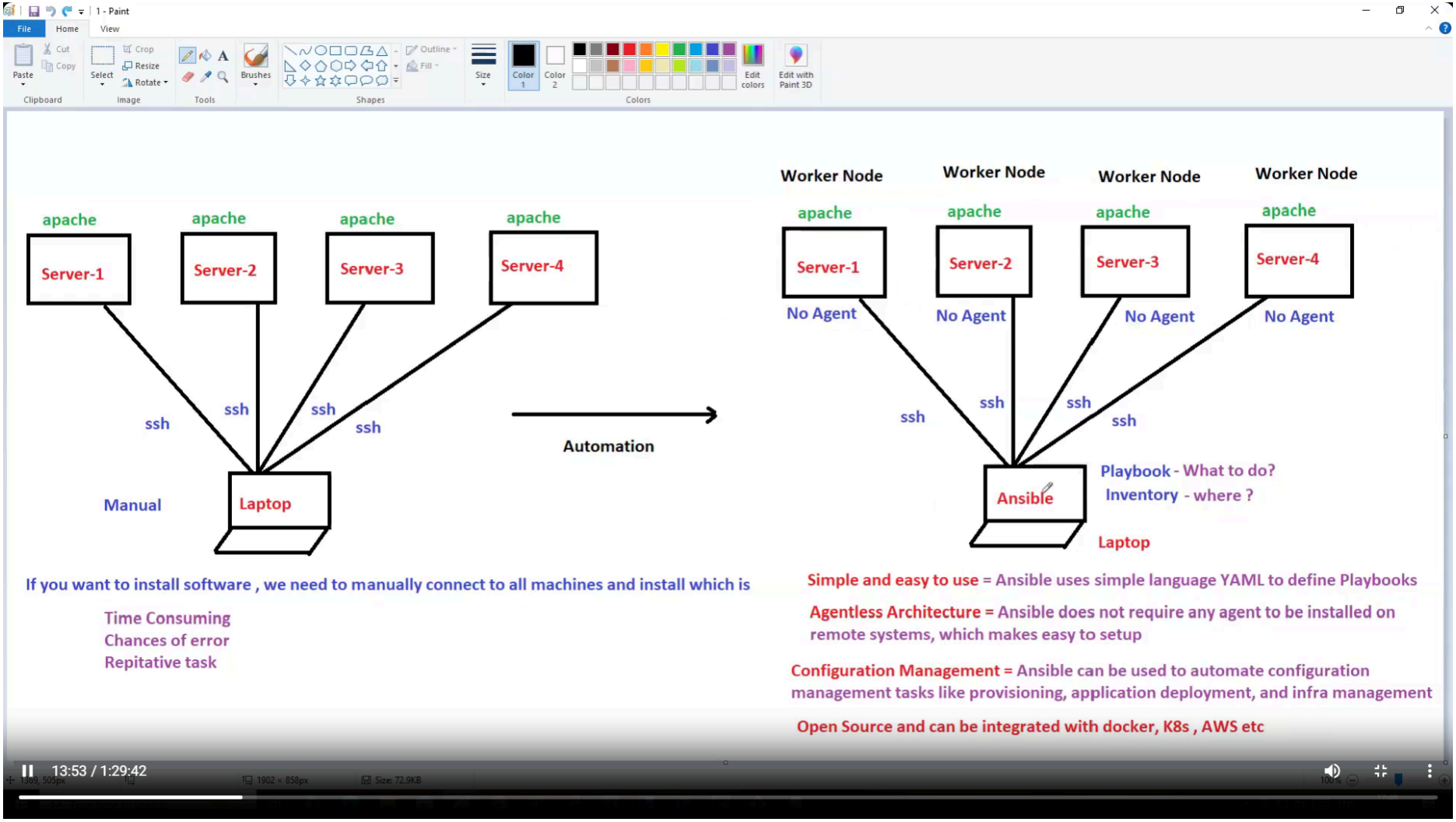
Integrates seamlessly with CI/CD tools like Jenkins, GitLab, and Azure DevOps.

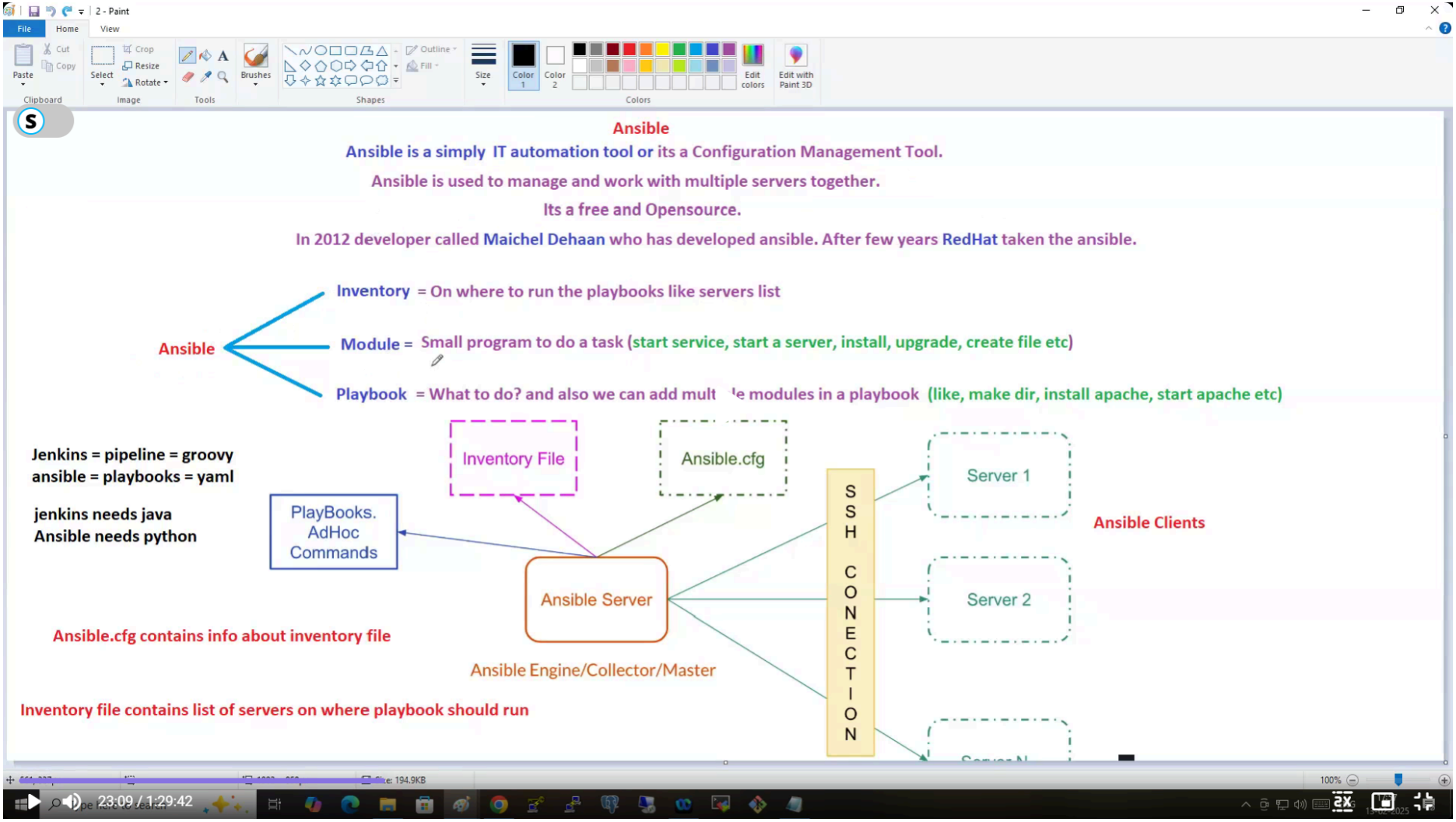
We can do Code Quality Analysis, Security Analysis

Architecture and Workflow

SonarQube Architecture:
SonarScanner: Scans the code and submits data.
SonarQube Server: Processes and analyzes data, hosts the UI.
Database: Stores code analysis reports and metrics.
Workflow:
Developer writes code.
Code is scanned using SonarScanner during build.
SonarQube analyzes the code and stores the results in the database.
Developers review issues via the SonarQube dashboard.

}


1 - Paint
File
Home
View
Paste
Cut
Copy
Select
Crop
Resize
Rotate
Brushes
Size
Color 1
Color 2
Edit colors
Edit with Paint 3D
Clipboard
Image
Tools
Shapes
Colors
apache
Server-1
Server-2
Server-3
Server-4
ssh
Manual
Laptop
Automation
Worker Node
No Agent
Playbook - What to do?
Inventory - where ?
Ansible
If you want to install software , we need to manually connect to all machines and install which is
Time Consuming
Chances of error
Repitative task
Simple and easy to use = Ansible uses simple language YAML to define Playbooks
Agentless Architecture = Ansible does not require any agent to be installed on remote systems, which makes easy to setup
Configuration Management = Ansible can be used to automate configuration management tasks like provisioning, application deployment, and infra management
Open Source and can be integrated with docker, K8s , AWS etc
13:53 / 1:29:42


2 - Paint
File
Home
View
Paste
Cut
Copy
Select
Crop
Resize
Rotate
Brushes
Outline
Fill
Size
Color 1
Color 2
Edit colors
Edit with Paint 3D
Clipboard
Image
Tools
Shapes
Colors
Ansible
Ansible is a simply IT automation tool or its a Configuration Management Tool.
Ansible is used to manage and work with multiple servers together.
Its a free and Opensource.
In 2012 developer called Maichel Dehaan who has developed ansible. After few years RedHat taken the ansible.
Ansible
Inventory = On where to run the playbooks like servers list
Module = Small program to do a task (start service, start a server, install, upgrade, create file etc)
Playbook = What to do? and also we can add mult le modules in a playbook (like, make dir, install apache, start apache etc)
Jenkins = pipeline = groovy
ansible = playbooks = yaml
jenkins needs java
Ansible needs python
Inventory File
Ansible.cfg
PlayBooks.
AdHoc
Commands
Ansible Server
SSH CONECTION
Server 1
Server 2
Ansible Clients
Ansible.cfg contains info about inventory file
Ansible Engine/Collector/Master
Inventory file contains list of servers on where playbook should run
100%
23:09 / 1:29:42

Ansible command to Install or run command on specific group of machines:

Development Group Machine Command:
ansible dev - a “yum install git“ -y

Production Group Machine Command:
ansible prod -a “yum install httpd” -y

Next Lecture : 14 Feb 2025 Notes here onwards:

Ansible Modules are units of code that can control system resources or execute system commands

MODULE
ansible all -m yum -a "name=git state=present" → -m is module, name = packagename, state = present (Install)

States:

yum modules (States for Package Management)
Present = install
Absent = uninstall
latest = install or upgrades to latest version

Services module (States for Service Management)

started = start
stopped = stop
restarted = restart the service
enabled = Ensures the service starts on boot.

Module command:

ansible all -m yum -a “name=git state=present”

Adhoc command to view git version - ansible all -a “git -v”

ansible all -m yum -a “name=maven state=present”

ansible all -m yum -a “name=httpd state=present”

Adhoc command to view apache version- ansible all -a “httpd -v”
ansible all -m service -a “name=httpd state=started”

Adhoc command to view status of httpd service: ansible all -a “systemctl status httpd”

ansible all -m yum -a “name=httpd state=absent”

ansible all -m yum -a “name=git state=absent”

ansible all -m yum -a “name=maven state=absent”

ansible all -m user -a “name=sanket state=present”

Adhoc cmd to view uesr: ansible all -a “cat /etc/passwd”

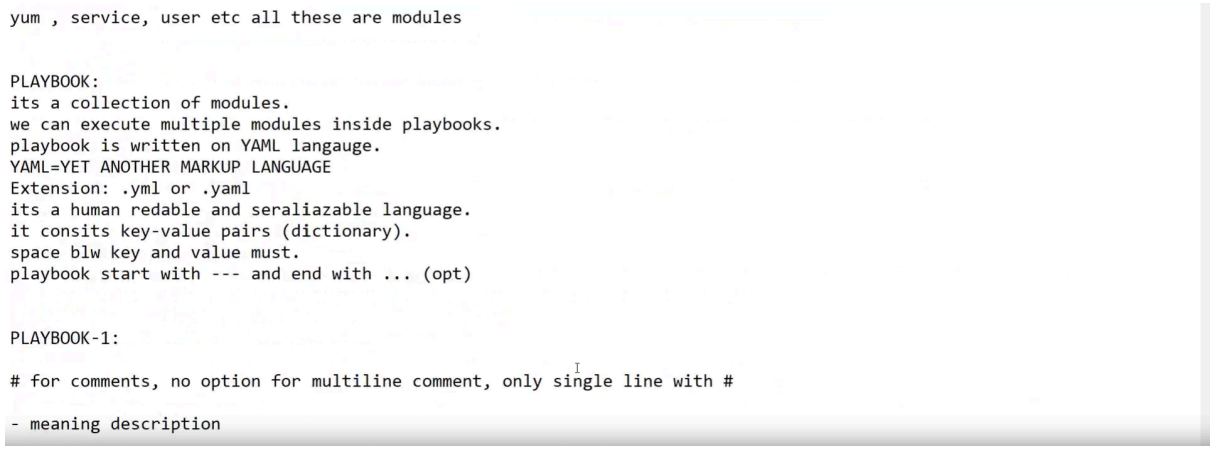
Adhoc cmd to create file: ansible all -a “touch file”

Touch python → created on master → you have to copied to all worker slaves then use cmd:

ansible all -m copy -a “src=python dest=/home/ec2-user/”

Adhoc command to view copied files in slave node: ansible all -a “ls /home/ec2-user/”

Adhoc command to remove all packages that were installed on slaves : ansible all -a “yum remove git* maven* httpd* -y”



Create playbook: vi pb1.yml

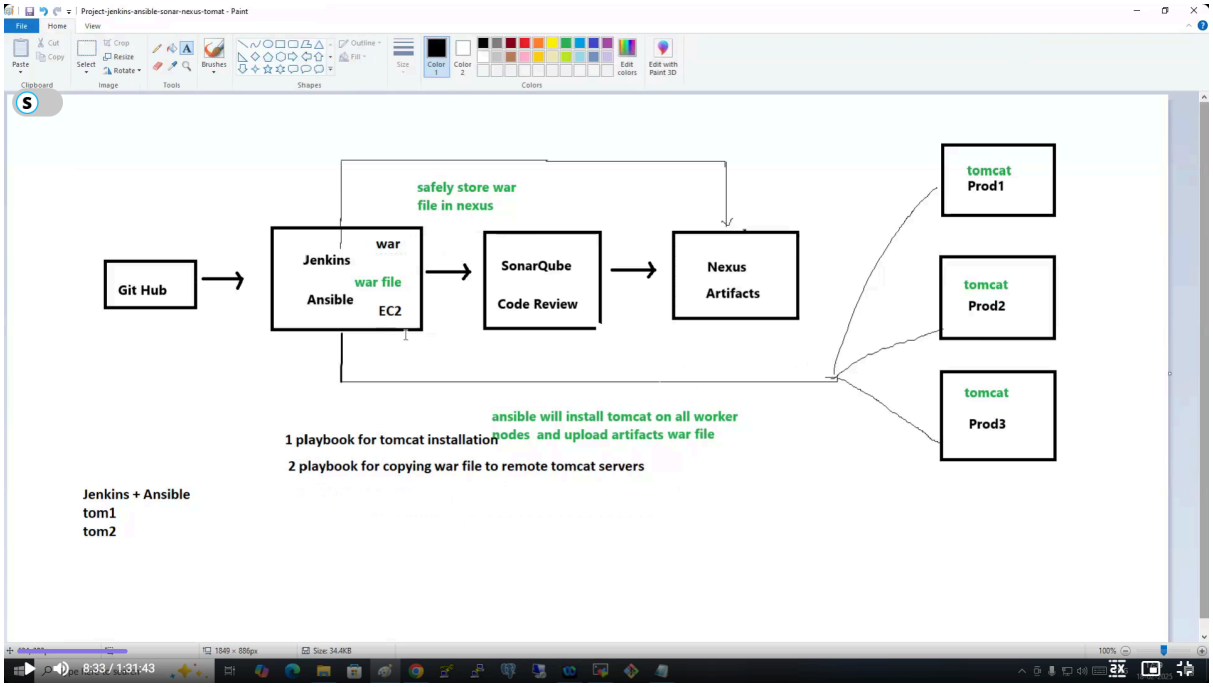
Playbook start with dash —- or …

To check the syntax of playbook we have command:

ansible-playbook - - syntax - check pb1.yml

Playbook code:

Run playbook using : ansible-playbook pb1.yml



Run the Pipeline before executing tomcat.yml → pipeline {

```
    agent any
    stages{
        stage('Checkout'){
            steps{
                git 'https://github.com/MrSanketPrajapatissp/java-project-maven-new.git'
            }
        }
        stage('Compile')
        {
            steps{
```

```
            sh 'mvn compile'
        }
    }
    stage('Test'){
        steps{
            sh 'mvn test'
        }
    }
    stage('Aritfact'){
        steps{
            sh 'mvn clean package'
        }
    }
    stage('Ansible Playbook'){
        steps{
            echo 'Deployed to Tomcat using playbook'
        }
    }
  }
}
```

Create pipeline name as AnsibleJenkinsPipelineProject

On Ansible + Jenkins Server:
Ansible-playbook tomcat.yml
cd /var/lib/jenkins/workspace/
ls
Vi deploy.yml —>
- name: Deploying war file to Tomcat Servers
hosts: all
tasks:

- name: Deploying war to Tomcats
copy:
src: /var/tib/jenkins/workspace/jenkinsansibtetomcatproject/target/myapp .war
dest: / root/ tomcat/webappsl

Ansible-playbook deploy.yml
This is manual deployment
Goto tomcat sea the app on browser on <url:EC2IP:8080>
using entering username:admin password:root123456
Now check the application is deployed now click on undeploy to automate manual process

Now go to jenkins manage plugins > Install Ansible plugins

Restart the jenkins server now next step is to configure ansible plugins go > manage jenkins > Tools – sea Ansible Installations add Installation path of ansible of Jenkins+ansible server who manaages worker nodes tom1 and tom2.
Click on Add :
Name: ansible
Path to ansible executables directory:
/bin
Click on Save

Now Jenkins need to talk to worker slaves this generated war file we have to put on worker slave automatically through pipelines:

Go to Manage Jenkins> Credentials > System > Global credentials (unrestricted)

Username: root
Password: root123 {here root passwd we configure at first step where we set all machine password: root123 , and did ssh-keygen on ansible+jenkins server and copy this key to ssh-copy-id }

ID;: linux credentials

Now copy deploy.yml to /etc/ansible → mv deploy.yml /etc/ansible

Now go to pipeline Syntax: Search-> ansible playbook

Playbook file path in workspace: /etc/ansible/deploy.yml

Inventory file path in workspace: /etc/ansible/hosts

Ssh connection : select linux credentials which we already created above

At last you sea click on Disable the host SSH key check

Click on Generate Pipeline Script → now copy the script → replace with ansible playbook stage : echo command

Run the pipeline: after job finish open tomcat1 tomcat 2 on browser to check the our app is deployed or not _=- yey it is deployed

New task: Launch EC2 instance from Ansible
Create IAM user and create keys
aws configure on Jenkins servers
install boto
-- > sudo pip install boto
vi ec2.ym1
- hosts: localhost
tasks:
- name: Launch EC2 instance
ec2:
region :
" ap-south-l"
count: 3
image:

" ami-dfg34kfdg"
instance _ type: "t2.micro"
ansible-playbook ec2.ym1
1
But, we use terraform not ansible in real time
Ansl
But,
e-p ay 00 ec . ym
we use terraform not ansible in real time
-- > tomcats on workernodes are in stop state, start it by modifying tomcat.yml and giving tag at last module
name: start the tomcat
shell: nohup / root/ tomcat/bin/startup.sh
tag: xyz
ansible-playbook tomcat.yml
- -tags xyz
--> to start tomcat service in all nodes

Name: server
choices parameter:
dev
test
prod
1
Name: server
save
run the pipeline
update the code in GitHub
--> and click

How to add PARAMETERS
- no need to give
The Project is parameterized
if string parameter -- multiple choices
'$server = prod'
on build now
host subset use parameters
choice
if you want to pass
host
all in pipeline
pass only single
run the build with parameters
In below pipeline code changed
to
for string parameters give dev, test
' $server '
limit:
previously it was limit:

DevOps Notes: A Structured Report and Learning Guide

This document is a structured compilation of notes covering core concepts and practical steps in Software Development Life Cycle (SDLC) and DevOps, focusing on tools like Git, Maven, Jenkins, and Ansible.1. Software Development Life Cycle (SDLC) & Methodologies1.1 What is SDLC?

The **Software Development Life Cycle (SDLC)** is the process of developing and delivering software projects.

SDLC Phases (8 Steps):

1. Requirement Gathering
2. Requirement Analysis
3. Design / Planning
4. Development
5. Testing
6. Deployment
7. Delivery
8. Maintenance

1.2 Waterfall Methodology

The Waterfall methodology was an earlier, linear approach used to develop and deliver projects.

- **Approach:** It is a linear methodology where everything happens step-by-step.
- **Flow:** Movement is only in the forward direction; a step must be completed before moving to the next.
- **Requirements & Budget:** Both requirements and budget are fixed.
- **Client Involvement:** Client involvement is very low, as the client typically sees the project only at the end.
- **Note:** Waterfall Methodology is not suitable for big projects.

1.3 Agile Methodology

Agile is an iterative approach to develop and deliver projects.

- **Approach:** Development and testing happen in parallel.
- **Delivery:** Projects are delivered in multiple releases, known as Sprints.
- **Feedback:** Client feedback is taken for every release.
- **Requirements & Budget:** Requirements and budget are not fixed, allowing for flexibility.
- **Frequency:** Project development, testing, and delivery are very frequent.
- **Client Involvement:** Client involvement is very high.
- **DevOps Connection:** Using DevOps culture makes adopting the Agile methodology very easy, as DevOps promotes Agile. Using

DevOps, we can achieve Continuous Integration (CI) and Continuous Deployment/Delivery (CD).

2. DevOps Fundamentals2.1 What is DevOps?

DevOps is a culture and a set of practices and processes used to deliver projects to the client quickly. It represents the collaboration between Development and Operations teams.

- **Goal:** To reduce the time of the Software Development Life Cycle.
- **Collaboration:** The Development team writes the code, and the Operations team deploys and delivers the project. Using the DevOps process, both teams work together to ensure the project is delivered within the estimated time.
- **Symbol:** The logo of DevOps represents "infinite" because it is a never-ending continuous process.
- **Note:** DevOps is not a one-person job; it is everyone's job in the project.

2.2 DevOps Advantages

1. Speed
2. Rapid Development
3. Quick Releases
4. Reliability
5. Security
6. Client Satisfaction
7. Teams Collaboration

2.3 DevOps Tools Overview

Category	Tools (Examples)
Build Tools	Ant, Maven, Gradle
Repository Tools	SVN, GitHub, BitBucket
Code Review Tools	PMD, Sonar Qube, Sonar lint

Code Deployment Tools	Jenkins, UDeploy
Containerization Tools	Docker
Orchestration Tools	Kubernetes
Configuration Tools	Chef, Ansible
Infrastructure as a Code (IaaC)	Terraform
Monitoring Tools	Nagios, Prometheus Grafana
Project Management Tools	Jira, Trello

3. Source Code Management: Git3.1 Git Commands Learned

Command	Description
`init`	Initializes a new Git repository
`add`	Stages changes
`commit`	Records changes to the repository
`log`	Shows commit history
`show`	Shows various objects (e.g., commit, tree)
`Restore`	Restores files from the index or a commit

3.2 Common Git Operations

- **Move files:** `mv -- *.html sanket/` (Moves all html files from current directory to sub directory)
- **Create files:**

    - `touch python (1..10)`
    - `touch hello.txt`
- **Stage all files:** `git add .`
- **Commit changes:** `git commit -m "python files"`
- **Stage a specific file:** `git add hello.txt`
- **View modified files:** `git status` (shows the file in red color)
- **Add remote repository (first time only):**
  git remote add origin https://github.com/ReyazShaik/testrepo.git
  This command is copied from GitHub and executes only the first time. Once executed, the remote origin is added to the `.git/config` file.
- **Check the remote repository details:** `git remote -v`
- **Push changes (local to central repo):**
  git push -u origin master
  This is used to commit the changes from local to the central repo.
- **Credential Caching (to avoid frequent password prompts):** `git config global credential.helper cache`
- **Remove credentials:** `git credential-cache exit`

[image]
[image]
[image]
[image][image][image][image]
[image]
[image]3.3 Generating SSH Key

1. Give the command into your terminal: `ssh-keygen`.
2. Copy the public key: `id_rsa.pub`.
3. Paste the public key into GitHub settings: Navigate to `path settings` -> `ssh and GPG keys` -> `Add new ssh key`, give it a name, and paste the public key there.
4. **Connecting using SSH:**

    - Change directory: `cd /home/ec2-user/`
    - Add remote repository: `git remote add origin git@github.com: ReyazShaik/test1.git`
    - Push changes: `git push -u origin master`

4. Build Automation: Maven4.1 Maven Overview

Maven is a build automation tool and project management tool.

- **Purpose:** It is used for the build purpose of a project.
- **Dependencies:** Maven downloads and installs necessary dependencies.
- **Configuration File:** Maven uses the `POM.xml` file, which stands for **Project Object Model**.

POM File Contents:

- Dependencies
- Plugins
- Configuration
- Project details

4.2 Maven Build Lifecycle and Phases

Build Phases (in order):

1. Validate
2. Compile
3. Test
4. Package
5. Install
6. Deploy

4.3 Artifacts

- **Java:** Basic raw code.
- **.class:** Executable file.
- **Jar:** Group of class files; works for the backend.
- **War:** Contains both Front-End (FE: html, css, js) and Back-End (BE: java) components.

[image]5. CI/CD Pipeline and Automation (Jenkins & Ansible)5.1 Jenkins Pipeline Examples

Basic Declarative Pipeline Structure:

```
pipeline {
    agent any {
        stages{
            stage('CLONE REPO GIT')
            {
                steps
                {
                    echo 'Cloning the git repository'
                }
            }
            stage('Build'){
                steps{
                    echo 'Building the application from code SCM'
                }
            }
            stage('Test')
            {
                steps{
                    echo 'Testing the applicatioin - SonarQube'
                }
            }
            stage('Code Review')
            {
```

```
            steps{
                echo 'Code Review Quality check - Code Review'
            }
        }
        stage('Artifacts')
        {
            steps{
                echo 'Pushing to Nexus / S3'
            }
        }
        stage('Deploy')
        {
            steps{
                echo 'Deploying the application to Tomcat'
            }
        }
    }
  }
}
```

Pipeline for Java Project with Maven and Ansible Deployment:

```
pipeline {
    agent any
    stages{
        stage('Checkout'){
            steps{
                git 'https://github.com/MrSanketPrajapatissp/java-project-maven-new.git'
            }
        }
        stage('Compile')
        {
            steps{
```

```
            sh 'mvn compile'
        }
    }
    stage('Test'){
        steps{
            sh 'mvn test'
        }
    }
    stage('Artifact'){
        steps{
            sh 'mvn clean package'
        }
    }
    stage('Ansible Playbook'){
        steps{
            echo 'Deployed to Tomcat using playbook' // To be replaced by Ansible command
        }
    }
  }
}
```

5.2 Ansible Fundamentals

Ansible Modules are units of code that can control system resources or execute system commands.

Commands on Specific Groups:

Group	Command	Action
Development	`ansible dev -a “yum install git“ -y`	Install git on 'dev' group machines

Production	`ansible prod -a “yum install httpd” -y`	Install httpd on 'prod' group machines

Module and States Reference:

Module	Purpose	States (Parameters)
`yum` (Package Management)	Install/Manage packages	`present` (install), `absent` (uninstall), `latest` (install or upgrades to latest version)
`service` (Service Management)	Manage services	`started` (start), `stopped` (stop), `restarted` (restart the service), `enabled` (starts on boot)

General Ad-hoc Command Examples:

- **Install Git (using module):** `ansible all -m yum -a “name=git state=present”`
- **View Git version:** `ansible all -a “git -v”`
- **Install Maven:** `ansible all -m yum -a “name=maven state=present”`
- **Install HTTPD:** `ansible all -m yum -a “name=httpd state=present”`
- **View Apache version:** `ansible all -a “httpd -v”`
- **Start HTTPD service:** `ansible all -m service -a “name=httpd state=started”`
- **View HTTPD status:** `ansible all -a “systemctl status httpd”`
- **Create user *sanket*:** `ansible all -m user -a “name=sanket state=present”`
- **View user list:** `ansible all -a “cat /etc/passwd”`
- **Copy file from master to slaves:** `ansible all -m copy -a “src=python dest=/home/ec2-user/”`
- **View copied files in slave node:** `ansible all -a “ls /home/ec2-user/”`
- **Remove multiple packages:** `ansible all -a “yum remove git* maven* httpd* -y”`

[image]5.3 Ansible Playbooks

- **Create Playbook:** `vi pb1.yml`
- **Start Playbook:** Playbooks start with `---` or `...`.
- **Check Syntax:** `ansible-playbook --syntax-check pb1.yml`
- **Run Playbook:** `ansible-playbook pb1.yml`

[image]5.4 Jenkins & Ansible CI/CD Automation

This section outlines automating the deployment of a `.war` file generated by Maven to Tomcat servers using Jenkins and Ansible.

1. Ansible Deployment Playbook (`deploy.yml`):

```
- name: Deploying war file to Tomcat Servers
  hosts: all
  tasks:
    - name: Deploying war to Tomcats
      copy:
        src: /var/tib/jenkins/workspace/jenkinsansibtetomcatproject/target/myapp .war
        dest: / root/ tomcat/webappsl
```

To run this manually, use: `Ansible-playbook deploy.yml`

2. Jenkins Configuration Steps for Automated Deployment:

1. **Install Plugin:** Install the Ansible plugin via `Manage Jenkins` > `Manage Plugins`.
2. **Configure Tools:** Configure the Ansible installation path via `Manage Jenkins` > `Tools`.
    - Name: `ansible`
    - Path to ansible executables directory: `/bin`
3. **Configure Global Credentials (for worker nodes):** Go to `Manage Jenkins` > `Credentials` > `System` > `Global credentials (unrestricted)`.
    - ID: `linux credentials`
    - Username: `root`

    - Password: `root123` (Assuming this password was configured across all machines)
4. **Move Playbook:** Copy `deploy.yml` to `/etc/ansible` on the Jenkins server: `mv deploy.yml /etc/ansible`.
5. **Generate Pipeline Script:** Use the Pipeline Syntax generator for the Ansible Playbook step.
    - Playbook file path: `/etc/ansible/deploy.yml`
    - Inventory file path: `/etc/ansible/hosts`
    - SSH Connection: Select the `linux credentials`
    - **Crucial Step:** Click on **Disable the host SSH key check**.
6. **Update Pipeline:** Copy the generated script and replace the `echo 'Deployed to Tomcat using playbook'` command in the `Ansible Playbook` stage of the pipeline.
7. **Run Pipeline:** Run the pipeline and check Tomcat servers (e.g., `url:EC2IP:8080`) to verify deployment.

5.5 Launching EC2 Instance from Ansible

Note: In real-time scenarios, Terraform is often used for Infrastructure as Code (IaaC) instead of Ansible for EC2 instance launch.

Setup Steps:

1. Create IAM user and keys.
2. Configure AWS credentials on Jenkins servers: `aws configure`.
3. Install Boto: `sudo pip install boto`.

Example Playbook (`ec2.yml`):

```
- hosts: localhost
  tasks:
    - name: Launch EC2 instance
      ec2:
        region : "ap-south-l"
        count: 3
        image: "ami-dfg34kfdg"
        instance _ type: "t2.micro"
```

Run command: `ansible-playbook ec2.ym1`5.6 Starting Tomcat Service with Tags

To start services on worker nodes that are in a stop state, modify the Ansible playbook (`tomcat.yml`) and use tags when running.

Modification to `tomcat.yml`:
- name: start the tomcat
  shell: nohup / root/ tomcat/bin/startup.sh
  tag: xyz
Run Command (to start the service on all nodes):
ansible-playbook tomcat.yml --tags xyz
5.7 Adding Parameters to Jenkins Pipeline

Parameters allow users to influence the build process, such as selecting a deployment environment (`dev`, `test`, `prod`).

Steps to Add Parameters:

1. Set the project as **Parameterized**.
2. Use a String or Choice parameter. For environment selection, provide choices like `dev, test, prod`.
3. **Use Parameter in Pipeline:** The `$server` parameter can be used in the pipeline to set the host limit.
    - Change the host limit from `limit: 'all'` to `limit: '$server'` for string parameters (e.g., `dev` or `test`).
4. Run the build with parameters.

6. ContainerizationDocker